

Cryptanalysis of Nonce Reuse in Encrypted RF Communications

Hassan Fares

September 2026

Abstract

This paper studies the exploitation of encrypted radio-frequency communications caused by nonce duplication. A simulation was implemented where two packets encrypted with AES-256-CTR using the same nonce were transmitted and intercepted in RetroSpectrum. Cryptanalysis was performed on both packets to reveal a critical relationship between the plaintext and ciphertext. A beam-search algorithm was then utilized to leverage this relationship and attempt to reveal the plaintext. Later, the algorithm proved to be successful and managed to recover the data within the encrypted messages. By recovering ciphertext-plaintext pairs, the AES-CTR keystream was also derived. This introduced an additional relationship that could potentially expose the AES key itself under certain weak key-derivation systems. The effective key space could be drastically reduced compared to the full AES-256 keyspace under these conditions. If an adversary manages to recover the key, the entire encrypted communication chain could be compromised.

1 Introduction

RF communication devices may store nonces and counters in persistent storage. Misconfiguration and poor management of cryptographic state can cause nonces or counters to repeat. This could be due to system resets, outages, counter regression, or other state-handling failures. An adversary who has recorded previous transmissions can exploit a later transmission if the same nonce is reused.

This paper demonstrates this scenario using AES-256-CTR-encrypted RF transmissions and evaluates how nonce reuse can enable plaintext recovery.

2 Overview

A simulated encrypted RF transmitter was used to test the nonce-reuse scenario. Two packets were transmitted using the same AES-256-CTR nonce and then RetroSpectrum [4] was used to intercept and decode the transmissions.

The transmitter encrypted two unique messages using the same nonce:

$$Nonce_1 = Nonce_2$$

The nonce extracted from each decoded packet was identical:

$$Nonce = \text{C3 32 25 A2 38 E5 11 24 9D 53 31 2F 62 69 95 A3}$$

2.1 Results

Packet ID	Recovered Nonce	Ciphertext Length
1	C3 32 25 A2 38 E5 11 24 9D 53 31 2F 62 69 95 A3	31 B
2	C3 32 25 A2 38 E5 11 24 9D 53 31 2F 62 69 95 A3	32 B

Repeated nonces

Table 1: Recovered nonce values from both intercepted packets.

Both transmissions reused the same nonce and contained different ciphertext.

2.2 Security Impact

For AES-CTR encryption [1]:

$$C = P \oplus KS(K, \text{Nonce})$$

Symbol	Definition
C	Ciphertext
P	Plaintext
K	Encryption key
$KS(K, \text{Nonce})$	Keystream generated from the key and nonce

A catastrophic relationship exists when the nonce is reused:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

An adversary that records both transmissions may exploit this relationship to reveal the plaintext without ever needing to know the encryption key.

3 Interception and Cryptanalysis with RetroSpectrum

3.1 Interception



Figure 1: Interception of two packets modulated with QPSK and encrypted with the same nonce

In this simulation, two packets were encrypted with the same nonce under AES-256-CTR and transmitted. From *Figure 1*, they were intercepted and by the constellation/IQ diagram, these two signals were QPSK modulated. (The four 90°-spaced phase states indicate QPSK modulation.)

3.2 Decode

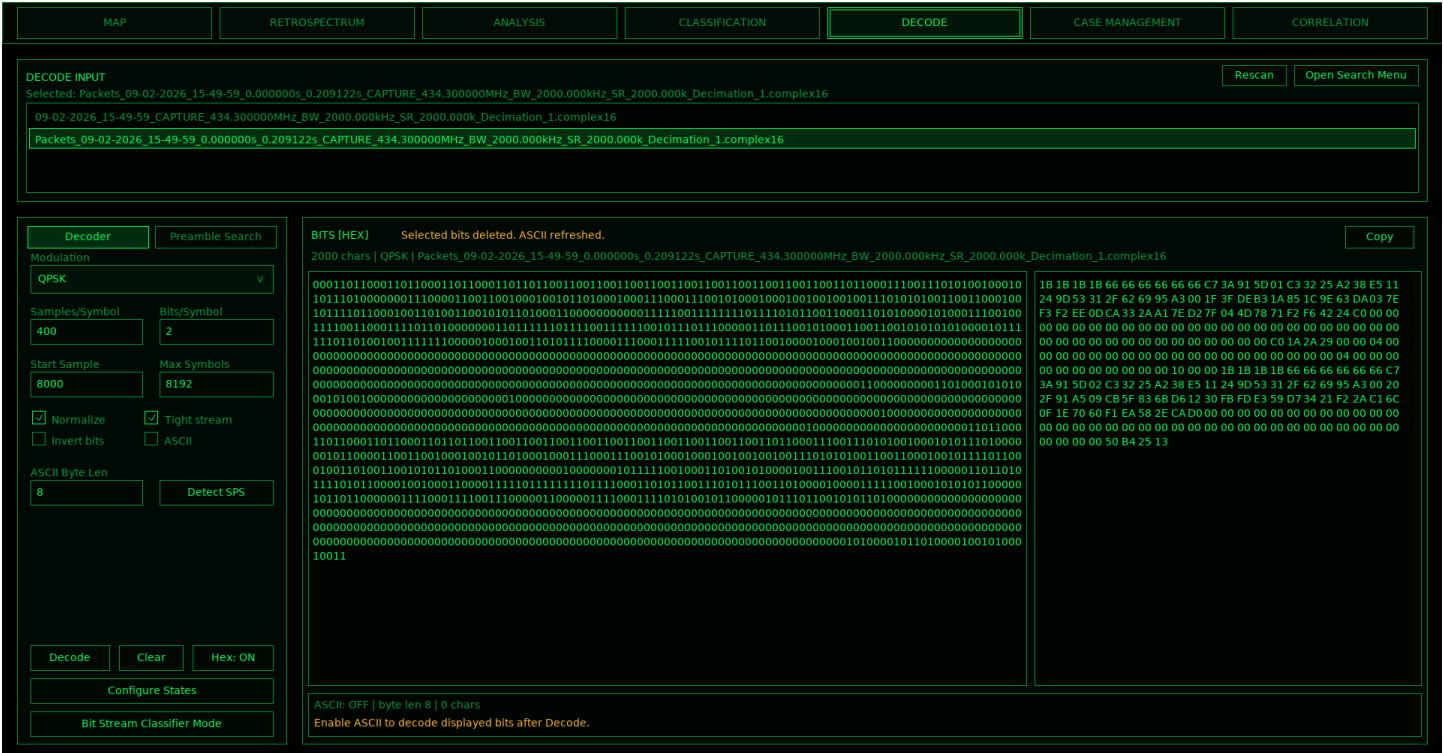


Figure 2: Decoded signal data from both packets.

RetroSpectrum demodulated the captured QPSK signal and recovered the packet bitstreams. The decoded fields included the synchronization data, packet ID, reused AES-CTR nonce block, ciphertext length, ciphertext, and CRC-32. The contents recovered are also shown in the right sidebar representing the bitstreams in hex.

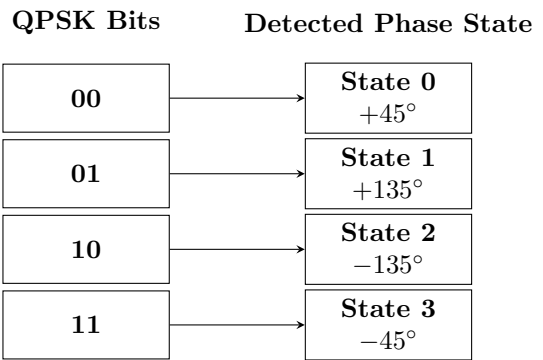


Figure 3: QPSK bit-to-phase-state mapping used to decode the packets within RetroSpectrum.

Training	Preamble	Sync	ID	Nonce	Length	Ciphertext	CRC-32
4 B	6 B	4 B	1 B	16 B	2 B	70 B	4 B

Table 2: Structure of each transmitted QPSK packet.

3.2.1 Packet 1

Field	Length	Recovered Value
Training	4 B	1B 1B 1B 1B
Preamble	6 B	66 66 66 66 66 66
Sync	4 B	C7 3A 91 5D
Packet ID	1 B	01
Nonce	16 B	C3 32 25 A2 38 E5 11 24 9D 53 31 2F 62 69 95 A3
Ciphertext Length	2 B	00 1F (31 bytes)
Ciphertext	31 B	3F DE B3 1A 85 1C 9E 63 DA 03 7E F3 F2 EE 0D CA 33 2A A1 7E D2 7F 04 4D 78 71 F2 F6 42 24 C0
Padding	39 B	00 repeated 39 times
CRC-32	4 B	C0 1A 2A 29

Table 3: Recovered fields from Packet 1.

3.2.2 Packet 2

Field	Length	Recovered Value
Training	4 B	1B 1B 1B 1B
Preamble	6 B	66 66 66 66 66 66
Sync	4 B	C7 3A 91 5D
Packet ID	1 B	02
Nonce	16 B	C3 32 25 A2 38 E5 11 24 9D 53 31 2F 62 69 95 A3
Ciphertext Length	2 B	00 20 (32 bytes)
Ciphertext	32 B	2F 91 A5 09 CB 5F 83 6B D6 12 30 FB FD E3 59 D7 34 21 F2 2A C1 6C 0F 1E 70 60 F1 EA 58 2E CA D0
Padding	38 B	00 repeated 38 times
CRC-32	4 B	50 B4 25 13

Table 4: Recovered fields from Packet 2.

3.3 Cryptanalysis

Because both packets were encrypted under AES-256-CTR using the same key and nonce, the same keystream was generated for both ciphertexts. This creates a keystream-reuse condition similar to a two-time pad, with a reuse depth of 2.

For the two intercepted ciphertexts:

$$C_1 = P_1 \oplus KS$$

$$C_2 = P_2 \oplus KS$$

XORing the ciphertexts cancels the shared keystream:

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

This does not directly reveal either plaintext, but it removes the AES keystream and exposes a relationship between the two original messages.

So how can we actually map the relationship between the two packets to reveal the plaintext? The answer lies in a computer science algorithm called beam search [2]. Rather than testing every possible plaintext combination, beam search keeps only the most likely candidates at each byte position, ranking them based on how closely they resemble valid English text.

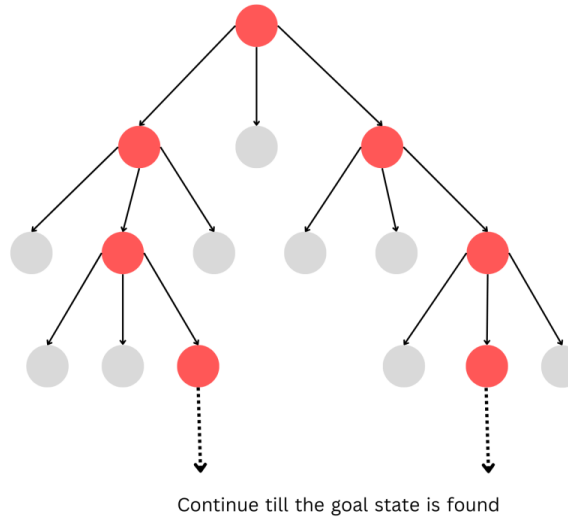


Figure 4: Beam search tree showing candidate-path selection [2].

3.3.1 Candidate Generation

For each byte position, the XOR relationship between the two plaintexts is already known:

$$X_i = C_{1,i} \oplus C_{2,i} = P_{1,i} \oplus P_{2,i}$$

This means that selecting a candidate byte for one plaintext determines the corresponding byte in the other plaintext:

$$P_{2,i} = P_{1,i} \oplus X_i$$

The search algorithm tests candidate byte pairs that satisfy this relationship. Candidates that do not produce valid alphabetic characters or spaces in both plaintexts are discarded. Spaces are treated as word boundaries and are only accepted when the preceding characters form a valid word.

3.3.2 Candidate Scoring

Each surviving candidate is scored according to the English words it forms. The algorithm uses the Brown corpus [3] as a dictionary and also constructs a set of valid word prefixes. This allows incomplete words to remain in the search as long as they could still evolve into a valid English word.

When a space is reached, the current word is checked against the dictionary. If the word is valid, its score is calculated as:

$$S(w) = |w|^2$$

Symbol	Definition
w	Candidate word being evaluated
$ w $	Number of characters in the word w
$S(w)$	Score assigned to the word w

For example, a three-character word contributes a score of 9, while a seven-character word contributes a score of 49. The scores of valid words from both plaintext candidates are added to the candidate's cumulative score. Candidates that cannot form a valid word or word prefix are discarded, while higher-scoring candidates are favored during beam selection.

3.3.3 Beam Selection

After all candidates for a byte position are scored, they are sorted by their cumulative score. Only the highest-scoring candidates are retained, with the number of surviving candidates determined by the selected beam width.

$$B_{i+1} = \text{Top}_W(\text{Expand}(B_i))$$

Symbol	Definition
B_i	Current beam of surviving candidates at position i
$\text{Expand}(B_i)$	All candidate extensions generated from the current beam
W	Selected beam width
$\text{Top}_W(\cdot)$	Operation that keeps the W highest-scoring candidates

Each surviving candidate is then expanded at the next byte position. This process continues until all ciphertext bytes have been evaluated, allowing the search to discard unlikely plaintext combinations without exhaustively testing every possible message.

3.3.4 Plaintext Recovery

The beam-selection process is repeated for each ciphertext byte until the full overlapping length of the two packets has been evaluated. At every position, the surviving candidates are expanded, scored, and reduced back to the selected beam width.

Once the final byte position is reached, the remaining candidates are ranked by their cumulative scores. The highest-scoring candidate represents the most likely pair of plaintext messages.

In this simulation, the highest-ranked candidate recovered:

$P_1 = \text{Your communications are exposed}$

$P_2 = \text{I can read all the transmissions}$

The AES-256 key was never recovered. Instead, plaintext recovery was possible because nonce reuse caused both ciphertexts to share the same keystream and this introduced a relationship between the ciphertexts and plaintexts.

3.3.5 Why Beam Search?

A brute-force search would require testing an exponentially growing number of plaintext combinations, approximately $O(k^n)$, where k is the number of allowed symbols and n is the message length.

Beam search reduces this to roughly $O(nWk)$, where W is the beam width, making the attack far more practical, but at the cost of potentially discarding the correct candidate early.

3.4 Known-Plaintext Key Recovery

Nonce reuse does not directly reveal the AES key. However, if a plaintext P and its ciphertext C are both known, the AES-CTR keystream can be recovered:

$$KS = C \oplus P$$

This keystream can then be used to verify candidate keys when the AES key is derived from a lower-entropy secret, such as a human-generated passphrase.

For example, a four-word candidate phrase could be converted into an AES-256 key using either:

$$K_c = \text{SHA256}(\text{candidate phrase})$$

or

$$K_c = \text{SHA512}(\text{candidate phrase})[0 : 32].$$

The candidate key can then generate a candidate CTR keystream:

$$KS_c = \text{AESCTR}(K_c, \text{Nonce}, 0^n)$$

If $KS_c = KS$, the candidate key is correct. A hash table could also store mappings such as

$$H(KS_c) \rightarrow \text{candidate phrase}$$

to allow fast lookup of candidate keystreams.

The main limitation is the size of the passphrase space. With 10,000 possible words, a four-word passphrase has:

$$10,000^4 = 10^{16} \approx 2^{53} \text{ possible combinations}$$

This is far smaller than the 2^{256} AES-256 key space, but it may still be impractical to exhaustively search.

If the key is recovered, the impact can extend beyond a single known message. A cryptanalyst could use that key with other nonces to decrypt additional communications encrypted under the same key.

Note that this scenario still requires weak key-derivation practices, such as deriving an AES key directly from a low-entropy passphrase using a fast hash function, specifically without a salt or a dedicated password-based key derivation function.

3.5 Limitations

This simulation assumed the plaintext contains only letters and spaces, which significantly reduced the search space. Allowing digits, punctuation, symbols, or arbitrary binary data would increase the number of candidate paths and make the search much more expensive.

The scoring also assumes natural English, so unusual words, abbreviations, or non-English plaintext may be ranked poorly or discarded.

4 Mitigations

- Ensure that nonces are never reused with the same encryption key.
- Store nonce and counter state in rollback-resistant persistent storage.
- Detect counter or nonce regression during device startup.
- Rotate encryption keys after a rollback, reset, or state-restoration event.
- Use monotonic counters that cannot be restored to an earlier value.
- Log transmitted nonce values so that reuse can be detected during operation.
- Reject or alert on repeated nonce values observed by the receiving system.
- Protect cryptographic state with integrity checks so unauthorized rollback can be detected.
- Design recovery procedures so that restoring an older system snapshot does not restore previously used cryptographic state.

5 Conclusion

This simulation demonstrated that nonce reuse in AES-256-CTR can introduce a dangerous relationship between the ciphertext and plaintext. RetroSpectrum was used to intercept and decode two QPSK packets that reused the same nonce, allowing the shared keystream relationship to be exploited.

Using cryptanalysis, the plaintext messages were recovered without recovering the AES encryption key itself. The results show that protecting persistent cryptographic state against rollback or reuse is critical for preventing duplicate nonces in stateful RF communication systems.

References

- [1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST Special Publication 800-38A, National Institute of Standards and Technology, December 2001. <https://doi.org/10.6028/NIST.SP.800-38A>
- [2] T. Dhondge, “Beam Search Algorithm With Logic and Implementation in Python,” AskPython, October 20, 2022. <https://www.askpython.com/python/examples/beam-search-algorithm>
- [3] NLTK Project, “Brown Corpus,” *Natural Language Toolkit: Accessing Text Corpora and Lexical Resources*. <https://www.nltk.org/book/ch02.html>
- [4] H. Fares, *RetroSpectrum*, GitHub repository, 2026. <https://github.com/hassanfarescodes/retrospectrum>